

L04 Robust estimation, and 3D sensing

Finding matches, turning them into cameras and 3D points, keeping that running on a moving robot, and a stereo rig that measures depth in metres.

Robust estimation, and 3D sensing

L04 · Unit 0

Before any geometry you need matches, and a third of them will be wrong. After that, a camera that moves can build its own map.

Wednesday ended with the geometry of two views and the question of which points can be matched at all. Today finds the matches, turns them into cameras and 3D points, keeps that running on a moving robot, and ends with a stereo rig, which measures depth directly.

Today

1. **Features** and **RANSAC**: finding matches when a third of them are wrong.
2. **Triangulation**: why two noisy rays need a least-squares answer.
3. **Structure from motion** and **bundle adjustment**.
4. **Visual SLAM**: keyframes and loop closure.
5. **Stereo**: depth in metres from a measured baseline.

Notes: Modules A4, A5 and A6, then A3. Lab 1 comes out on Friday.

Invariance

what a feature has to survive

So far every transformation was computed from correspondences we were given. Finding them needs a definition first: an **invariance** is a property that some operation leaves unchanged. You have used one all term:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \sim k \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad k \neq 0$$



A feature has to survive scale, rotation, translation, illumination and geometric distortion such as an affine warp, and still hand back the same point.

A feature transform

A feature transform $x = f(I)$ maps information to a summary of part of that information, in a different form: an image in, a vector out. A vision-language model is the same shape of object, with different features falling out of the end.

which is short for

SIFT	scale invariant feature transform
SURF	speeded up robust features
ORB	oriented FAST and rotated BRIEF
BRIEF	binary robust independent elementary features

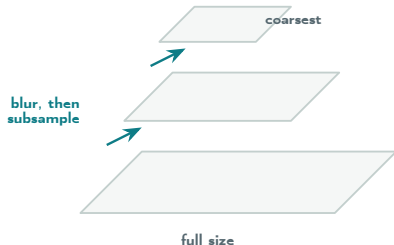


Image: OpenCV SIFT tutorial, Apache-2.0

Each buys a different subset of those invariances for a different amount of compute. ORB is still what you find inside real-time SLAM.

Image pyramids

where scale invariance comes from



- Blur, subsample, repeat. Each blur is a **local average**, so each level keeps only what survives at a coarser scale.
- A feature found at several levels is one whose appearance does not depend on how far away you were standing.
- **Gaussian** and **Laplacian** pyramids are two choices of filter.

So the invariance is obtained by construction rather than assumed.

Matching is a distance

A feature is a vector of numbers, so “is this the same point?” becomes “how close are these two vectors?” Choose a metric, choose a tolerance:

$$\text{distance}(f_1, f_2) < \varepsilon \implies \text{call it a match}$$

- An **inlier** is a pair closer than ε and an **outlier** is a pair that is not. Matching hands you plenty of both.
- Brute force compares every feature against every other, and two things go wrong: the features do not land on the same spot of the object in both images, and the region the two images share does not hold the same number of them.
- Two fixes. Sample densely enough that some pair does land close, and estimate in a way that rejects outliers. The second one is RANSAC, after one cheap filter.

The ratio test

the cheap filter before RANSAC

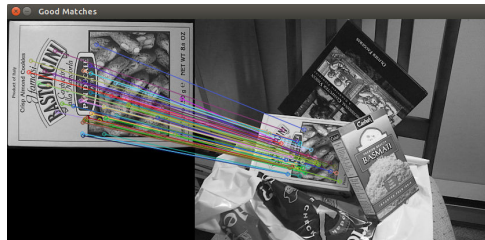
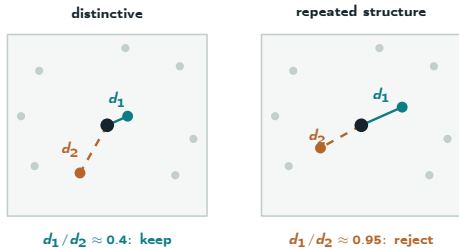
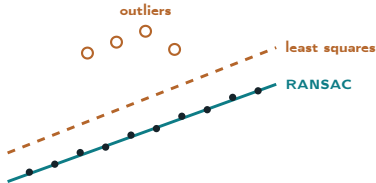


Image: OpenCV FLANN matcher tutorial, Apache-2.0

- Keep a match only when the best distance is under about 0.8 of the second best. Repeated structure, where two candidates look alike, is exactly what it throws away.
- Mutual consistency is the other cheap filter: a 's best match is b , and b 's best match is a .

Why least squares dies here

After the ratio test you might have 60–80% correct matches. The rest are wrong by an arbitrary amount, which is a different problem from noise.



- A squared cost weights a residual by its square, so one bad match at 200 px outweighs a hundred good ones at 1 px.
- No threshold on the residual fixes this, because there are no residuals until there is a fit.

Chicken and egg: you need the model to spot the outliers, and you need the outliers gone to fit the model.

RANSAC

1. Draw a **minimal** sample, the smallest set that determines the model: 8 for F, 5 for E, 3 for PnP, 4 for a homography.
2. Fit the model to that sample exactly.
3. Count **inliers**: data within a threshold of the model.
4. Keep the best hypothesis, and repeat.
5. Refit on all inliers of the winner.

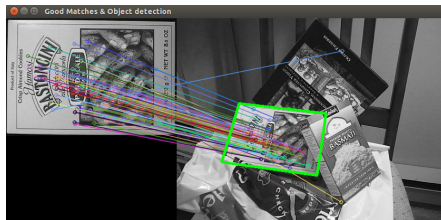


Image: OpenCV feature homography tutorial, Apache-2.0

The method needs one clean sample in N tries. How good the samples are on average does not matter.

How many iterations?

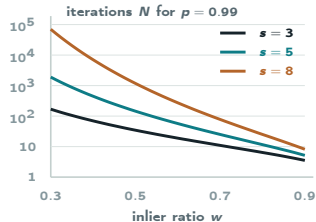
■ Derive N from the probability of never drawing a clean sample

N does not depend on the number of data points.

What that costs you

$$p = 0.99$$

Inlier ratio w	$s = 3$ (PnP)	$s = 5$ (E)	$s = 8$ (F)
0.8	7	12	26
0.6	19	57	272
0.5	35	146	1177
0.4	70	448	7025
0.3	169	1893	70188



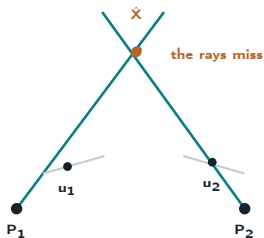
Read the last column. The minimal sample size sits in an exponent, which is the entire argument for the five-point algorithm.

Making RANSAC work in practice

- **Adaptive N :** you do not know w in advance. Re-estimate it from the best inlier count so far and shrink N as you go, usually an order of magnitude cheaper.
- **The threshold matters most:** too tight and every hypothesis has few inliers; too loose and outliers get counted. Set it from your measurement noise, about 3σ in pixels.
- **MSAC** scores by truncated squared residual rather than a binary count, which breaks ties between hypotheses with the same inlier set.
- **LO-RANSAC** refits locally on a promising hypothesis before scoring. Cheap, and consistently better.

Triangulation

the rays do not meet



You know the two cameras P_1, P_2 and a correspondence $u_1 \leftrightarrow u_2$. You want the world point X that projects onto both, where π divides out the last coordinate:

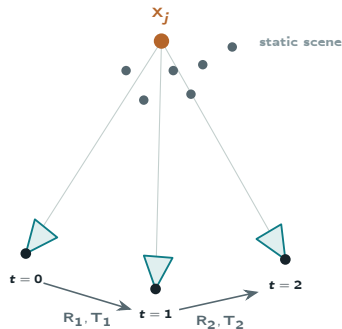
$$\min_{\tilde{X}} \sum_{i=1}^2 \|\pi(P_i \tilde{X}) - u_i\|^2$$

With noise in either measurement the two back-projected rays miss each other, so there is nothing to intersect. Start from a linear estimate, the DLT, then minimise this reprojection error, and check that the point has positive depth in both cameras.

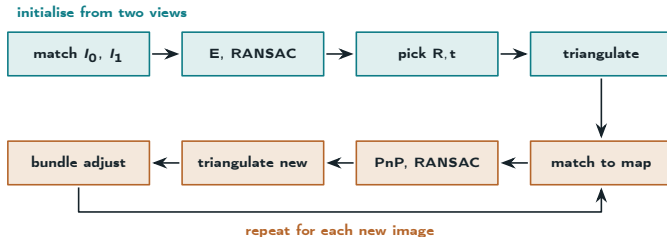
Structure from motion

Two or more views of a static scene, taken by one camera that moved. The data are only the images.

- **Known:** the images, and the intrinsics K , the same at every time step.
- **Unknown:** every camera $P_t = K[R_t | T_t]$, which changes because the camera moves, and every 3D point X_j .
- Two views make a stereo pair with shared intrinsics, except that nobody measured the baseline.



Incremental structure from motion

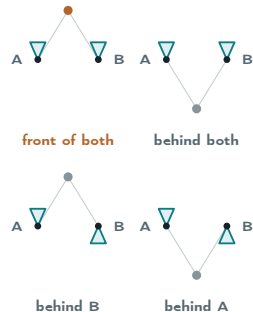


Set the first camera as the origin. Initialise from two views, then add one image at a time: PnP places it, triangulation adds its new points, and bundle adjustment refines both.

What you already know changes the order. If some 2D–3D correspondences are known, from a marker or an earlier map, PnP can come first; IMU odometry can seed every pose.

Initialising from two views

Recover R and t from E



Adding an image with PnP

Reminder: PnP

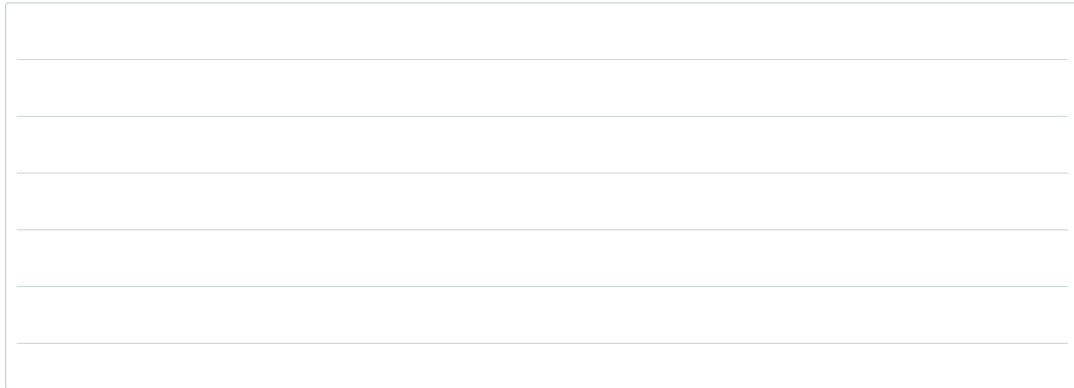
Perspective-n-Point: you know some 3D points and the pixel where each one appears in a single image, and you want the camera pose R, t . Triangulation holds the cameras and solves for a point; PnP holds the points and solves for a camera.

- Match the new image to features whose 3D points already exist. Each match is a 2D–3D correspondence.
- Solve PnP inside RANSAC. Three correspondences give up to four poses, and a fourth picks one.
- Refine the pose by minimising reprojection error with the points held fixed.
- Triangulate matches that now have two registered views, and check their cheirality.

Adding an image with PnP, drawn

by hand

■ Draw the map so far, the new camera, its 2D–3D matches, and the new points it lets you triangulate

A large rectangular box with a thin black border, containing six horizontal light blue lines for drawing. The lines are evenly spaced and extend across the width of the box.

Bundle adjustment

everything at once

$$\arg \min_{\{\mathbf{T}_i\}, \{\mathbf{X}_j\}} \sum_i \sum_{j \in \mathcal{V}(i)} \rho \left(\left\| \pi(\mathbf{K}, \mathbf{T}_i, \mathbf{X}_j) - \mathbf{u}_{ij} \right\|_{\Sigma_{ij}}^2 \right)$$

- Every camera pose and every 3D point, against every observation, in one nonlinear least-squares problem.
- $\mathbf{K}$ is shared; only $\mathbf{R}_t$ and $\mathbf{T}_t$ change from frame to frame.
- ρ is a robust kernel such as Huber, so a surviving outlier bends the answer instead of breaking it.

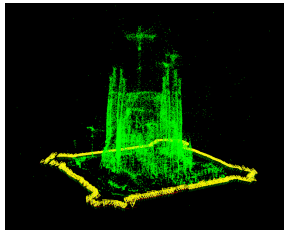
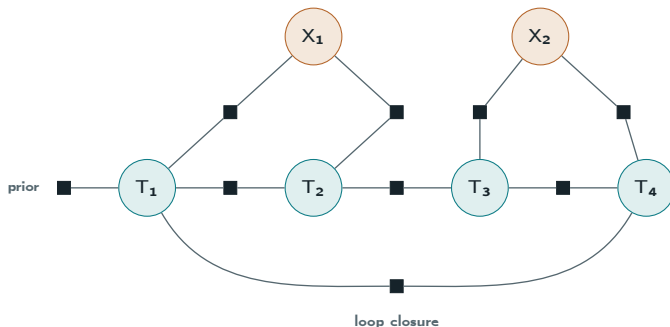


Image: OpenCV contrib sfm module docs, Apache-2.0

The same thing as a graph



- **Circles** are unknowns: poses T_i and landmarks X_j . **Squares** are factors, one per measurement.
- Reading this one: a prior pins T_1 , odometry links neighbouring poses, each landmark links the poses that saw it, and a loop closure links T_1 to T_4 .

Factor graphs, one square at a time

A factor graph draws an estimation problem. Each circle is something you do not know. Each square is something you measured or assume, and it touches only the circles that measurement depends on.

Factor	Connects	Its residual is small when
Prior	one pose	the pose sits where we put the origin
Odometry	two consecutive poses	their relative motion matches what the wheels or the IMU measured
Projection	one pose, one landmark	the landmark projects onto the pixel where it was seen
Loop closure	two distant poses	the two frames agree that they saw the same place

Solving the graph means choosing every circle so that all the squares are satisfied together, in the least-squares sense: $\sum_k \|r_k\|_{\Sigma_k}^2$, one term per square. A new sensor adds a new kind of square; the solver stays the same.

Structure from motion, odometry, SLAM

	Input	Output	How it runs
Structure from motion	a set of images, in any order	poses and 3D points	offline, all at once
Visual odometry	a video stream	the camera trajectory	online, frame to frame
Visual SLAM	a video stream	the trajectory and a map it reuses	online, with keyframes and loop closure

Odometry chains relative poses, so small errors add up into **drift**. Nothing inside odometry can remove it. SLAM removes it by recognising a place it has already mapped.

Visual SLAM, step by step

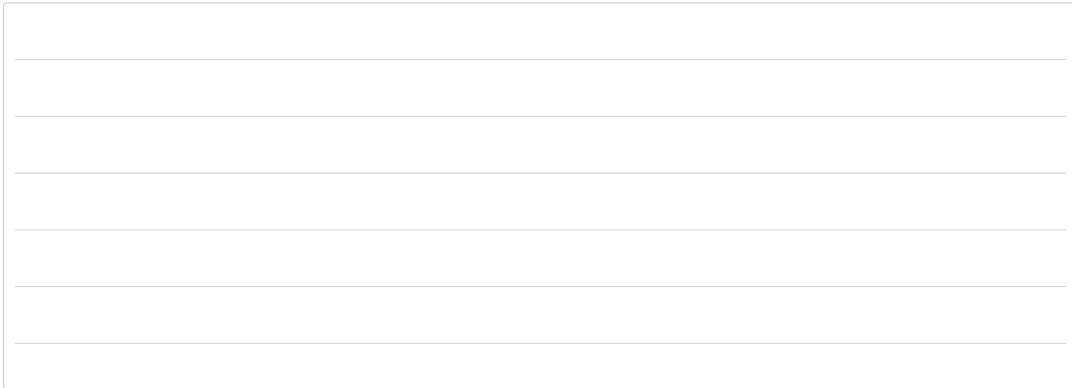
1. Set the first frame as the origin.
2. Run structure from motion on the first frames: correspondences, triangulation, PnP.
3. Bundle adjust to refine the poses and points.
4. Add a **keyframe** when the view has changed enough that too few correspondences overlap.
5. Keep doing structure from motion relative to the current keyframe.
6. Detect loops: compare each new keyframe with the earlier ones. A match means the same 3D point seen again.
7. At every loop, bundle adjust over the keyframes to remove the drift.

Noise gets handled at three scales: SVD inside the geometry, bundle adjustment inside structure from motion, and bundle adjustment across keyframes.

Visual SLAM, drawn

by hand, 1 of 2

■ Draw the camera moving around a loop: keyframes, the points they share, and the drift



A large rectangular box with a thin black border, containing six horizontal lines for drawing. This area is intended for a hand-drawn diagram illustrating a camera moving around a loop in Visual SLAM, showing keyframes, shared points, and drift.

Visual SLAM, walked through

by hand, 2 of 2

■ Walk the steps over the drawing: track, add a keyframe, detect the loop, correct

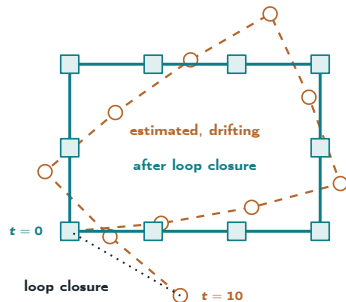
Tracking and local mapping

- **Tracking, every frame:** predict the pose from the last motion, project nearby map points into the image, match them, and solve for this frame's pose alone.
- **Mapping, every keyframe:** triangulate new points against neighbouring keyframes, then bundle adjust a local window of keyframes and their points while older keyframes stay fixed.
- **When to add a keyframe:** for example when the share of map points still tracked falls below a threshold, or when many frames have passed since the last one.

Keyframes keep bundle adjustment small enough to run while the robot is still moving.

Loop closure

- **Recognise the place:** describe each keyframe as a histogram of visual words, which are quantised descriptors, and look for an earlier keyframe with a similar histogram.
- **Verify it:** match features between the two keyframes and fit their relative pose with RANSAC. Keep the loop only if enough inliers agree.
- **Correct it:** add the loop as a constraint, optimise the pose graph, then bundle adjust. Past keyframes move too.



Stereo: triangulation, bolted down

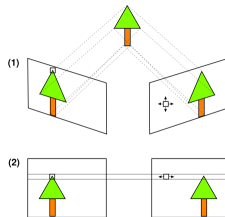
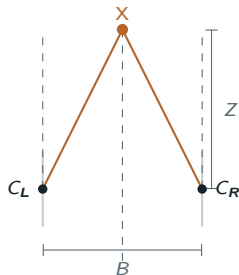


Diagram: Allan Ortiz, public domain, via

Wikimedia Commons

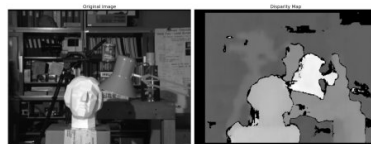


Image: OpenCV depth map tutorial, Apache-2.0

Rectify, so epipolar lines become rows and matching is a search along one row. Then $Z = fB/d$ with disparity $d = x_L - x_R$. For example, $f = 500$ px, $B = 1$ m and $d = 100$ px give $Z = 5$ m.

What you should take away

- Triangulation, PnP and bundle adjustment all minimise reprojection error; a linear solve only starts them.
- Structure from motion is a loop: place an image with PnP, triangulate what is new, bundle adjust.
- Bundle adjustment refines every pose and every point against every observation, in one sparse least-squares problem.
- Visual SLAM adds keyframes to keep that problem small and loop closure to remove drift.
- Stereo gives metric depth because its baseline is measured in metres; monocular structure from motion has no scale until something supplies it.

Notes: Modules A4, A5, A6 and A3. Szeliski §7 for features, §11 for structure from motion and bundle adjustment, §12 for stereo.

Next time

L05 · How a neural network learns

Wednesday opens with the depth sensors we did not reach: LiDAR scans, and metric versus relative depth. Then the other half of the toolkit, starting from zero: what a neural network is and how it is trained.

- One neuron, a loss, and one step of gradient descent worked by hand.
- Activation functions, layers, and softmax for classification.
- Training in practice: batches, validation, overfitting, and normalisation.
- Convolutional networks, and the encoder–decoder shape of a depth network.

Lab 1 is out on Friday, Sep 18. Run the fetch script the day it comes out, not the night before it is due. **Both diagnostic quizzes are due Sun Sep 27.** Quiz 2 is today's depth-sensor material and Wednesday's.